Explicação do Código – Semáforo em Assembly AVR

[Nome(s) do grupo]

19 de outubro de 2026

Sumário

1 Visão geral do projeto **3**
1.1 Fluxo geral de execução . . . 3

2 Registradores e constantes **3**
2.1 Constantes gerais . . . 4

3 Inicialização no `reset` **4**
3.1 Pilha e interrupções . . . 4
3.2 Configuração das portas . . . 4

4 Tabela de estados dos LEDs na Flash **5**
4.1 Subrotina `charge_leds` . . . 5

5 Temporização com Timer1 **6**
5.1 Modo CTC, clock e prescaler . . . 6
5.2 Tabela `qtd_runs` . . . 6
5.3 Interrupção `OCI1A_Interrupt` . . . 6

6 Multiplexação dos displays com Timer0 **7**
6.1 Tempo e modo CTC . . . 7
6.2 Inicialização e vetor . . . 7
6.3 Interrupção `TIMER0_Interrupt` . . . 8
6.4 Subrotina `charge_display` . . . 8

7 Comunicação serial USART **8**
7.1 Configuração de 9600 baud e 8N1 . . . 8
7.2 Mensagens e ponteiros na Flash . . . 9
7.3 Subrotina `charge_serial` . . . 9
7.4 Interrupção `USART_UDRE_Interrupt` . . . 9

8 Proteção dos registradores nas interrupções **9**

9 Sequência dos 12 estados **10**
9.1 Comportamento específico da contagem exibida . . . 10

10 Testes na simulação e montagem física **10**
10.1 Observações de revisão do código . . . 11

11 Conclusão 11

1 Visão geral do projeto

O programa implementa uma máquina de estados para um cruzamento com quatro semáforos veiculares (S1 a S4) e um semáforo de pedestres (S5), totalizando 14 LEDs. A lógica é executada em Assembly AVR no microcontrolador ATmega328P, com frequência definida no código como 16 MHz. Além da iluminação, há dois displays de sete segmentos acionados por sinais BCD e uma saída serial que informa a situação de cada semáforo.

A implementação utiliza três mecanismos de interrupção: o Timer1 controla a duração dos estados e a contagem regressiva; o Timer0 alterna rapidamente os dois displays; e a USART transmite cada caractere quando o registrador de dados está disponível. As configurações dos LEDs, os tempos e as mensagens ficam em tabelas na memória Flash.

1.1 Fluxo geral de execução

1. O vetor de reset desvia para `reset`, que prepara a pilha e inicializa as variáveis de controle.
2. São configuradas a USART e as direções de entrada/saída das portas; depois são carregados LEDs, texto serial e duração do primeiro estado.
3. Os displays recebem inicialmente os dígitos 4 e 6. Timer1 e Timer0 são configurados, e `sei` habilita as interrupções globais.
4. O Timer0 multiplexa os dígitos; o Timer1 atualiza a contagem regressiva, avança os estados e alterna o LED vermelho de pedestres no estado correspondente.
5. A USART envia o status por interrupções `UDRE`. O laço principal não consulta flags e apenas permanece em espera.

```
1 reset:
2     ; inicializacoes e carregamentos
3     rcall init_timer
4     rcall init_timer0
5     sei
6 loop:
7     jmp loop
```

2 Registradores e constantes

Os nomes definidos por `.def` representam registradores físicos de uso geral do AVR, não posições de RAM alocadas dinamicamente.

Nome / registrador	**Função no programa**
`state_count` / r16	Índice do estado atual, de 0 até 11.
`serial_high` / r17	Byte alto do endereço, em bytes, do próximo caractere serial.
`serial_low` / r18	Byte baixo do endereço da mensagem serial.
`temp` / r19	Auxiliar para máscaras, endereçamento e acesso aos periféricos.

Nome / registrador	Função no programa
`temp2` / r20	Auxiliar para bits dos LEDs e envio dos dígitos BCD.
`count_aux` / r21	Número de interrupções de Timer1 ainda restantes, usando o valor da tabela menos um.
`dezena` / r22	Dígito decimal das dezenas mostrado no display.
`unidade` / r23	Dígito decimal das unidades mostrado no display.
`display_select` / r24	Alterna entre 0 e 1 para escolher o display multiplexado.
`s1count` / r25	Conta duas interrupções de 0,5 s para formar um segundo.

2.1 Constantes gerais

`state_limit = 12` determina a quantidade de estados; `s1_limit = 2` indica que dois eventos de Timer1 equivalem a um segundo. As constantes `d46/u46`, `d64/u64` e `d75/u75` guardam pares de dígitos (46, 64 e 75), carregados em situações determinadas pelo programa. `CLOCK` define a frequência de 16 MHz usada nos cálculos de temporização e da USART.

3 Inicialização no `reset`

3.1 Pilha e interrupções

A pilha deve existir antes de qualquer `rcall`, `push` ou interrupção, pois esses mecanismos salvam endereços de retorno ou registradores. O programa escreve o endereço final da SRAM, `RAMEND`, nos registradores `SPH` e `SPL`. Depois inicializa os contadores e configura os periféricos. O `sei` só aparece ao final, evitando que uma interrupção ocorra durante a configuração incompleta.

```
ldi temp, high(RAMEND)
out SPH, temp
ldi temp, low(RAMEND)
out SPL, temp
clr state_count
clr display_select
clr s1count
```

A sequência atual configura primeiro a USART e as portas com `init_uart` e `init_ports`. Depois chama `charge_leds`, `charge_serial` e `charge_runtimes`. Por fim, carrega 46 nos dígitos e executa as inicializações de Timer1 e Timer0.

3.2 Configuração das portas

```
ldi temp, $3F
out DDRB, temp
out DDRC, temp
ldi temp, $FD
out DDRD, temp
```

`DDRB = 0x3F` e `DDRC = 0x3F` configuram como saída os bits 0 a 5 das portas B e C. `DDRD = 0xFD` configura PD0 e PD2–PD7 como saídas, deixando PD1 sob controle da USART para transmissão. O significado dos pinos no projeto é:

Pinos	**Destino**
PB0–PB2	S1: verde, amarelo, vermelho
PB3–PB5	S2: verde, amarelo, vermelho
PC0–PC2	S3: verde, amarelo, vermelho
PC3–PC5	S4: verde, amarelo, vermelho
PD6–PD7	S5: verde e vermelho
PD2–PD5	Quatro linhas BCD dos displays
PD0	Seleção entre os displays
PD1	TX da USART

Os dois displays compartilham as linhas BCD. Apenas o sinal de seleção determina qual recebe o valor enviado naquele instante. A polaridade real depende do inversor/transistores usados na montagem.

4 Tabela de estados dos LEDs na Flash

As diretivas `.equ` associam cada LED a uma posição de bit. Por exemplo, `S1_G = (1 << 0)` representa o verde do primeiro semáforo e `S1_Y = (1 << 1)` representa o amarelo. Os 14 sinais ocupam os bits de 0 a 13 de uma palavra de 16 bits; os bits 14 e 15 não são usados.

```
.equ S1_G = (1 << 0)
.equ S1_Y = (1 << 1)
.equ S1_R = (1 << 2)
; ... ate S5_G (bit 12) e S5_R (bit 13)
mapeamento:
    .dw S1_R|S2_R|S3_R|S4_R|S5_R
    .dw S1_R|S2_R|S3_R|S4_R|S5_G
```

Cada `.dw` armazena uma palavra com as luzes ativas naquele estado. O operador `|` combina as máscaras. Se o bit de uma luz vale 1, aquela saída será acionada; se vale 0, ficará desativada. Em particular, o **bit 1** representa `S1_Y`, o LED amarelo do semáforo 1.

4.1 Subrotina `charge_leds`

1. Z aponta para o início de `mapeamento` em Flash. O `« 1` transforma o endereço em palavras usado pelo montador em endereço em bytes, exigido por `lpm`.
2. `state_count` é multiplicado por dois com `lsl`, porque cada estado ocupa dois bytes. `add` e `adc` ajustam os bytes baixo e alto de Z, incluindo eventual transporte.
3. `lpm temp, Z+` lê o primeiro byte; `lpm temp2, Z` lê o segundo. A máscara `0x3F` seleciona os seis bits enviados a `PORTB`.
4. Dois pares `lsl temp`/`rol temp2` deslocam o conjunto de 16 bits duas posições para a esquerda. Os seis bits baixos de `temp2` passam então a representar as luzes destinadas à `PORTC`.

5. `andi temp2, 0xC0` conserva os dois bits restantes, que correspondem a PD6 e PD7. A rotina lê `PORTD`, preserva PD0–PD5 com `0x3F` e combina o resultado com esses dois LEDs antes de escrever novamente.

```
lpm temp, Z+
lpm temp2, Z
push temp
andi temp, $3F
out PORTB, temp
pop temp
lsl temp
rol temp2
lsl temp
rol temp2
; ... PORTC, mascara $C0 e escrita em PORTD
```

Dessa maneira o desenho lógico de cada estado é mantido em Flash, e a rotina apenas o converte para as portas físicas.

5 Temporização com Timer1

5.1 Modo CTC, clock e prescaler

Timer1 possui 16 bits e opera em CTC: o contador é comparado com `OCR1A`, e a comparação gera um evento de interrupção. O código usa $f_{CPU} = 16\,\text{MHz}$, prescaler 256 e período solicitado de 500 ms.

$$T_1 = \frac{(\text{OCR1A} + 1) \cdot 256}{16\,000\,000}.$$

Logo, TOP $= \frac{16\,000\,000 \cdot 0{,}5}{256} - 1 = 31\,249$, dentro da faixa de 16 bits. O valor é separado em `high(TOP)` e `low(TOP)` e escrito em `OCR1AH` e `OCR1AL`.

`TCCR1A` e `TCCR1B` ativam o modo `WGM=0b0100`; os bits `CS12:CS10=100` selecionam o prescaler 256. `OCIE1A` em `TIMSK1` habilita a interrupção de comparação A. A constante `TOP` é fixa nesta versão: a rotina não recalcula nem recarrega `OCR1A` a cada estado.

5.2 Tabela `qtd_runs`

A versão atual não possui uma tabela `tops`: todos os estados usam a mesma interrupção de 0,5 s. A duração muda pela quantidade de eventos contados em `count_aux`.

```
qtd_runs:
    .db 1, 23, 7, 3, 43, 7, 1, 119, 7, 1, 49, 7
```

Ao carregar um estado, `charge_runtimes` lê o byte na posição `state_count` com `lpm count_aux, Z`. Como a interrupção testa o zero *antes* de decrementar, um valor inicial n resulta em $n + 1$ interrupções no estado. Exemplo: valor 23 corresponde a 24 eventos de 0,5 s, isto é, 12 s.

5.3 Interrupção `OCI1A_Interrupt`

A cada 0,5 s, o Timer1 executa duas tarefas interligadas. Primeiro incrementa `s1count` para acompanhar os segundos dos displays; depois trata a duração do estado atual. Quando `s1count = 2`, a unidade diminui em um. Se a unidade já for zero, a dezena diminui e a

unidade recebe 9, produzindo, por exemplo, a passagem de 40 para 39. `s1count` é zerado nos caminhos de decrementação.

```
inc s1count
cpi s1count, s1_limit
brne continua
tst unidade
breq decrementa_dezena
; ... decrementa unidade ou dezena
```

Em seguida, se `count_aux` ainda não for zero, ele é decrementado. No estado de índice 2 (terceiro estado), `sbi PIND,7` inverte o nível de PD7 a cada 0,5 s, produzindo o pisca-pisca vermelho dos pedestres. A escrita de 1 em um bit de `PIND` alterna o bit correspondente da saída `PORTD` no ATmega328P.

Quando `count_aux` já está em zero no início da interrupção, `state_count` é incrementado e volta a zero ao atingir 12. A nova configuração dos LEDs e a mensagem serial são carregadas, o contador `TCNT1` é zerado e `charge_runtimes` prepara a duração do próximo estado. Nos índices 7 e 9, o código também carrega pares de dígitos específicos (64 e 75) e reinicia `s1count`.

Comportamento particular desta versão. Quando dezena e unidade são zero, o fluxo desvia para `continua` sem zerar `s1count`. Por isso, após chegar a 00, esse contador pode ultrapassar 2 e só voltará a ser zerado quando um dos pares de dígitos for recarregado. Isso é um detalhe de implementação a considerar em futuros ajustes, não uma alteração aplicada à documentação.

6 Multiplexação dos displays com Timer0

6.1 Tempo e modo CTC

Timer0 possui 8 bits, logo `OCR0A` comporta valores entre 0 e 255. O programa define `delay0=2` ms, prescaler 1024 e `WGM0=0b10`, correspondendo ao modo CTC. O cálculo inteiro produz TOP0 = 30:

$$\mathrm{TOP0} = \mathrm{int}\Big(\frac{16\,000\,000 \cdot 2}{1024 \cdot 1000} - 1\Big) = 30, \quad T_0 = \frac{(30+1)\cdot 1024}{16\,000\,000} = 1{,}984\,\mathrm{ms}.$$

Portanto, nesta versão a alternância acontece aproximadamente a cada 1,984 ms, **não** a cada 5 ms, apesar de um comentário antigo dizer 5 ms.

6.2 Inicialização e vetor

`init_timer0` grava 30 em `OCR0A`, habilita `OCIE0A` em `TIMSK0`, configura `TCCR0A` para CTC e ativa o clock dividido por 1024 em `TCCR0B`. O vetor `OC0Aaddr` desvia para `TIMER0_Interrupt`. A interrupção usa `display_select` e não interfere diretamente na contagem de estados do Timer1.

```
ldi temp, TOP0
out OCR0A, temp
lds temp, TIMSK0
ori temp, (1 << OCIE0A)
sts TIMSK0, temp
```

```
ldi temp, (WGM0 << WGM00)
out TCCR0A, temp
ldi temp, (prescale0 << CS00)
out TCCR0B, temp
```

6.3 Interrupção `TIMER0_Interrupt`

O `eor display_select, temp`, com `temp=1`, alterna somente o bit menos significativo do seletor. Quando ele vale 1, a rotina coloca PD0 em 1 e copia a dezena para `temp2`. Quando vale zero, limpa PD0 e copia a unidade. Em ambos os casos chama `charge_display`; ao final, restaura os registradores e retorna com `reti`.

```
ldi temp, 1
eor display_select, temp
in temp, PORTD
tst display_select
breq mostra_unidade
; mostra_dezena: PD0=1, temp2=dezena
; mostra_unidade: PD0=0, temp2=unidade
```

Cada display é selecionado uma vez a cada dois eventos, ou aproximadamente a cada 3,968 ms (cerca de 252 atualizações por segundo por display, no relógio ideal configurado).

6.4 Subrotina `charge_display`

A máscara binária `11000011` preserva os bits PD7, PD6, PD1 e PD0, apagando os bits PD2–PD5. Duas instruções `lsl temp2` deslocam o dígito BCD quatro fios para as posições PD2–PD5; depois `or` une o BCD aos demais pinos e `out PORTD,temp` atualiza o hardware.

```
charge_display:
    andi temp, 0b11000011
    lsl temp2
    lsl temp2
    or temp, temp2
    out PORTD, temp
    ret
```

A multiplexação apresenta alternadamente dezenas e unidades por um único barramento BCD. É necessário que o circuito externo ligue um display e desligue o outro conforme o nível de PD0.

7 Comunicação serial USART

7.1 Configuração de 9600 baud e 8N1

A USART trabalha em modo assíncrono, sem paridade, com 8 bits de dados e 1 bit de parada (8N1). A taxa nominal é 9600 baud, com divisão normal por 16:

$$\text{UBRR} = \text{int}\left(\frac{16\,000\,000}{16 \cdot 9600} - 1\right) = 103.$$

`init_uart` escreve o valor em `UBRR0H/UBRR0L`, configura `UCSR0C` com `UCSZ01=1` e `UCSZ00=1` e liga `TXEN0` em `UCSR0B`. O sinal de saída é TX no pino PD1. O programa transmite, mas não habilita recepção serial.

7.2 Mensagens e ponteiros na Flash

Existem 12 mensagens, `msg1` a `msg12`, descrevendo em texto a cor de S1 a S5. Cada texto é gravado por `.db` e termina com o byte nulo 0, que marca seu fim. Os números 13 (CR) e 10 (LF) produzem quebras de linha no terminal. A tabela `serial_mensage` possui 12 entradas `.dw`, cada uma contendo o endereço de uma mensagem.

```
msg1:
    .db "S1: Vermelho", 13, 10, ... , 0, 0
serial_mensage:
    .dw msg1
    .dw msg2
    ; ... ate msg12
```

Em AVRASM, rótulos de código/Flash usados nessa tabela são endereços de palavra; a leitura de bytes posterior usa a conversão adequada.

7.3 Subrotina `charge_serial`

A subrotina posiciona Z no início da tabela de endereços, multiplica `state_count` por dois para selecionar a entrada de 16 bits e carrega os dois bytes do ponteiro em `serial_low` e `serial_high`. Depois desloca o ponteiro de palavra para byte com `lsl`/`rol` e ativa `UDRIE0` em `UCSR0B`. Essa habilitação inicia o envio por interrupção quando `UDR0` estiver vazio.

7.4 Interrupção `USART_UDRE_Interrupt`

A interrupção copia `serial_low/high` para Z, lê um caractere da Flash com `lpm temp,Z+` e salva o endereço atualizado. `tst temp` verifica se o byte lido é zero: caso positivo, `UDRIE0` é desabilitado, impedindo interrupções desnecessárias. Caso contrário, o byte é escrito em `UDR0`, para transmissão. Assim, cada interrupção transfere um caractere, e a USART chama a rotina novamente quando pode receber outro.

```
lpm temp, Z+
tst temp
breq fim_serial
sts UDR0, temp
; ...
fim_serial:
    lds temp, UCSR0B
    andi temp, ~(1 << UDRIE0)
    sts UCSR0B, temp
```

8 Proteção dos registradores nas interrupções

Uma interrupção pode ocorrer durante a execução de outra parte do programa. Para não destruir valores em uso, a rotina salva com `push` os registradores auxiliares e o `SREG` (que contém flags como Zero e Carry). A restauração usa `pop` na ordem inversa, respeitando o funcionamento LIFO da pilha. A instrução `reti` retorna da interrupção e restabelece a aceitação normal de novas interrupções.

Na `USART_UDRE_Interrupt` são protegidos `temp`, `SREG`, `ZH` e `ZL`. Na `TIMER0_Interrupt` são protegidos `temp`, `SREG` e `temp2`. Na `OCI1A_Interrupt` também são preservados os

dois registradores Z, já que as rotinas de carregamento os utilizam. Registros como `state_count`, `s1count`, `dezena` e `unidade` não são restaurados porque a atualização deles faz parte do resultado desejado da interrupção.

9 Sequência dos 12 estados

A tabela abaixo deriva diretamente das entradas de `mapeamento` e `qtd_runs`. As durações são calculadas por (valor da tabela + 1) × 0,5 s. A notação V/A/R significa verde/amarelo/vermelho e S5 representa pedestres.

Estado	Cores acionadas	Tempo	Serial
1	S1–S5: R	1 s	msg1
2	S1–S4: R; S5: V	12 s	msg2
3	S1–S4: R; S5: R piscante	4 s	msg3
4	S1–S5: R	2 s	msg4
5	S1: V; S2: V; S3–S5: R	22 s	msg5
6	S1: V; S2: A; S3–S5: R	4 s	msg6
7	S1: V; S2–S5: R	1 s	msg7
8	S1: V; S2: R; S3: R; S4: V; S5: R	60 s	msg8
9	S1: A; S2: R; S3: R; S4: A; S5: R	4 s	msg9
10	S1–S5: R	1 s	msg10
11	S1: R; S2: R; S3: V; S4: R; S5: R	25 s	msg11
12	S1: R; S2: R; S3: A; S4: R; S5: R	4 s	msg12

O ciclo dos 12 estados totaliza **140 s** (2 min e 20 s), considerando o clock ideal e as durações configuradas. O LED S5 vermelho é alternado por software no terceiro estado, apesar de a máscara base desse estado indicar vermelho acionado.

9.1 Comportamento específico da contagem exibida

O programa começa exibindo 46. Ao entrar no estado de índice 7 (estado 8), carrega 64; ao entrar no índice 9 (estado 10), carrega 75. Fora desses pontos, o Timer1 continua decrementando os mesmos dois registradores uma vez por segundo, quando o contador intermediário alcança 2. Assim, o valor do display não é reiniciado a cada mudança de estado. Os números 46, 64 e 75 são valores de programação encontrados na versão enviada e não devem ser confundidos automaticamente com a duração de cada estado da tabela.

10 Testes na simulação e montagem física

- **Clock configurado:** 16 MHz, tanto no cálculo das interrupções quanto no divisor da USART. É necessário conferir se a frequência efetiva do ATmega328P no SimulIDE ou na placa coincide com essa configuração.
- **Serial:** taxa 9600, formato 8N1; PD1 (TX) deve ser ligado ao RX do terminal, com referência de terra compartilhada.
- **LEDs:** confirmar os 12 padrões de cores, especialmente o pisca-pisca de S5 durante o estado 3, e comparar as transições com a tabela de tempos.
- **Displays:** conferir as quatro linhas BCD PD2–PD5, a seleção PD0 e o funcionamento dos transistores/inversor. O Timer0 atualiza o seletor aproximadamente a cada 1,984 ms.

- **Mensagens:** conferir se o texto de cada `msg1`–`msg12` aparece na mudança correspondente de estado, sem caracteres extras.
- **Validação:** este documento descreve o código fornecido. Não constitui comprovação de compilação bem-sucedida, teste físico completo ou medição real da temporização.

10.1 Observações de revisão do código

A análise identificou diferenças entre alguns comentários e o código executável: o comentário de Timer0 menciona 5 ms, mas `delay0=2`; os comentários sobre prescaler em Timer1 citam `0b101` em um trecho, embora a configuração efetiva seja `0b0100`; e a tabela atual não usa mais `tops`. Esses pontos foram descritos conforme as instruções efetivas, sem alterar o arquivo Assembly original. Uma verificação futura também deve observar que `s1count` não é zerado no caminho em que ambos os dígitos já são zero.

11 Conclusão

O programa organiza o cruzamento como uma máquina de 12 estados. Cada estado é representado de forma compacta por uma palavra de 16 bits na Flash, enquanto outra tabela determina sua duração em eventos de 0,5 s. Timer1 faz a progressão dos semáforos e a contagem regressiva; Timer0 mantém os displays atualizados por multiplexação; e a USART envia textos de status sem consulta constante a flags no laço principal. A separação entre inicialização, tratamento de interrupções, rotinas de carregamento e tabelas facilita manutenção e depuração.